Workflow

```
Images/Screenshots ~ /Users/kaustubhgupta/Desktop/E-comm/.venv
```

Import data

```
# ==========================
# 基础数据处理
# ==========================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as ticker
import matplotlib.dates as mdates
import warnings
warnings.filterwarnings('ignore')
plt.rcParams['font.sans-serif'] = ['Arial']
plt.rcParams['axes.unicode_minus'] = False

import pandas as pd, numpy

# 查看版本
print(pd.__version__, numpy.__version__)

# 设置显示选项
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

# 读取数据
df = pd.read_csv('olist_orders_dataset.csv')

# 基本信息
print("数据形状:", df.shape)
print("\n列名:", df.columns.tolist)
df.head()
```

```
2.2.0 1.26.4
数据形状: (99441, 8)

列名: <bound method Index.tolist of Index(['order_id', 'customer_id', 'order_status',
       'order_purchase_timestamp', 'order_approved_at',
       'order_delivered_carrier_date', 'order_delivered_customer_date',
       'order_estimated_delivery_date'],
      dtype=object)>
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	2017-10-18 00:00:00
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c7b7336d	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13 00:00:00
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 10:00:35	2018-08-08 13:50:00	2018-08-17 18:06:29	2018-08-23 00:00:00
3	949d5b44dbf5de918fe9c16f97b7d964	f88197965ea6438d896869a483dd4fb0	delivered	2018-08-18 19:06:01	2018-08-19 03:44:43	2018-08-20 13:32:00	2018-08-28 15:52:27	2018-09-04 00:00:00
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7b2089	delivered	2018-08-13 08:59:02	2018-08-13 09:45:35	2018-08-14 13:30:00	2018-08-19 14:02:17	2018-09-05 00:00:00

Explore data

```
# 查看数据类型
print("数据类型:")
df.dtypes

# 查看缺失值
print("缺失值:")
missing = df.isnull().sum()
missing[missing > 0]

# 查看唯一值数量
print("唯一值数量:")
df.nunique()

# 数值型变量统计
print("数值型变量统计:")
df.describe()

# 分类变量统计
print("分类变量统计:")
df.describe(include=['object'])
```

```
数据类型:
order_id                       object
customer_id                    object
order_status                   object
order_purchase_timestamp       object
order_approved_at              object
order_delivered_carrier_date   object
order_delivered_customer_date  object
order_estimated_delivery_date  object
dtype: object
缺失值:
order_approved_at              160
order_delivered_carrier date     1
order_delivered_customer_date 2965
dtype: int64
唯一值数量:
order_id                       99441
customer_id                    99441
order_status                       8
order_purchase_timestamp       98875
order_approved_at              90733
order_delivered_carrier date   81018
order_delivered_customer_date  95834
order_estimated_delivery_date    459
dtype: int64
数值型变量统计:
分类变量统计:
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier date	order_delivered_customer_date	order_estimated_delivery_date
count	99441	99441	99441	99441	99281	98966	96476	99441
unique	99441	99441	8	98875	90733	81018	95834	459
top	73c2643a0a0468b6a0ac2cb0a9f2f36b	3d3d1a2f671d1a9fa0cb7f85c2d7f386	delivered	2018-04-11 10:48:14	2018-02-27 04:04:16	2018-05-09 15:48:00	2017-05-16 00:00:00	2017-12-20 00:00:00
freq	1	1	96478	3	9	47	3	1652

Data cleaning

```
# 查看原始数据
print("原始数据形状:", df.shape)

# 处理时间列
time_columns = ['order_purchase_timestamp', 'order_approved_at', 'order_delivered_carrier date', 'order_delivered_customer date', 'order_estimated_delivery date']
for col in time_columns:
    df[col] = pd.to_datetime(df[col])

# 检查缺失值
print("缺失值:")
missing = df.isnull().sum()
missing[missing > 0]

# 处理缺失值
df['order_approved_at'].fillna(df['order_purchase_timestamp'], inplace=True)
df['order_delivered_carrier date'].fillna(df['order_estimated_delivery date'], inplace=True)
df['order_delivered_customer date'].fillna(df['order_estimated_delivery date'], inplace=True)

# 删除重复值
print("重复值数量:", df.duplicated().sum())
df = df.drop_duplicates()

# 检查异常值
print("异常值检查:")
print("订单状态分布:")
print(df['order_status'].value_counts)

# 检查时间逻辑
print("时间逻辑检查:")
print("购买时间晚于审核时间的订单数:", (df['order_purchase_timestamp'] > df['order_approved_at'].dt.date).sum())
print("审核时间晚于发货时间的订单数:", (df['order_approved_at'] > df['order_delivered_carrier date'].dt.date).sum())
print("发货时间晚于送达时间的订单数:", (df['order_delivered_carrier date'] > df['order_delivered_customer date'].dt.date).sum())

# 保存处理后的数据
df.to_csv('processed_orders.csv', index=False)
print("处理后数据已保存")
```

```
原始数据形状: (99441, 8)
缺失值:
order_approved_at              160
order_delivered_carrier date     1
order_delivered_customer date 2965
dtype: int64
重复值数量: 0
异常值检查:
订单状态分布:
order_status
delivered      96478
shipped         1107
canceled         625
unavailable      609
invoiced         314
processing       301
created            5
approved           2
Name: count, dtype: int64
时间逻辑检查:
购买时间晚于审核时间的订单数: 0
审核时间晚于发货时间的订单数: 0
发货时间晚于送达时间的订单数: 0
处理后数据已保存
```

User behavior analysis

```
# 查看处理后的数据
print("处理后数据形状:", df.shape)
print("\n列名:", df.columns.tolist)

# 基本统计信息
print("数值型变量统计:")
df.describe()

# 分类变量统计
print("分类变量统计:")
df.describe(include=['object'])

# 时间范围分析
print("时间范围:")
print("最早订单时间:", df['order_purchase_timestamp'].min())
print("最晚订单时间:", df['order_purchase_timestamp'].max())

# 订单状态分布
print("订单状态分布:")
print(df['order_status'].value_counts)
```

```
处理后数据形状: (99441, 8)

列名: <bound method Index.tolist of Index(['order_id', 'customer_id', 'order_status',
       'order_purchase_timestamp', 'order_approved_at',
       'order_delivered_carrier date', 'order_delivered_customer date',
       'order_estimated_delivery date'],
      dtype=object)>
数值型变量统计:
分类变量统计:
时间范围:
最早订单时间: 2016-09-04 21:15:19
最晚订单时间: 2018-10-17 17:30:18
订单状态分布:
order_status
delivered      96478
shipped         1107
canceled         625
unavailable      609
invoiced         314
processing       301
created            5
approved           2
Name: count, dtocs: int64
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier date	order_delivered_customer date	order_estimated_delivery date
count	99441	99441	99441	99441	99281	98966	96476	99441
unique	99441	99441	8	98875	90733	81018	95834	459
top	73c2643a0a0468b6a0ac2cb0a9f2f36b	3d3d1a2f671d1a9fa0cb7f85c2d7f386	delivered	2018-04-11 10:48:14	2018-02-27 04:04:16	2018-05-09 15:48:00	2017-05-16 00:00:00	2017-12-20 00:00:00
freq	1	1	96478	3	9	47	3	1652

Explore data

```
# 查看数据类型
print("数据类型:")
df.dtypes

# 查看缺失值
print("缺失值:")
missing = df.isnull().sum()
missing[missing > 0]

# 查看唯一值数量
print("唯一值数量:")
df.nunique()

# 数值型变量统计
print("数值型变量统计:")
df.describe()

# 分类变量统计
print("分类变量统计:")
df.describe(include=['object'])
```

```
数据类型:
order_id                       object
customer_id                    object
order_status                   object
order_purchase_timestamp       object
order_approved_at              object
order_delivered_carrier date   object
order_delivered_customer date  object
order_estimated_delivery date  object
dtype: object
缺失值:
order_approved_at              160
order_delivered_carrier date     1
order_delivered_customer date 2965
dtype: int64
唯一值数量:
order_id                       99441
customer_id                    99441
order_status                       8
order_purchase_timestamp       98875
order_approved_at              90733
order_delivered_carrier date   81018
order_delivered_customer date  95834
order_estimated_delivery date    459
dtype: int64
数值型变量统计:
分类变量统计:
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier date	order_delivered_customer date	order_estimated_delivery date
count	99441	99441	99441	99441	99281	98966	96476	99441
unique	99441	99441	8	98875	90733	81018	95834	459
top	73c2643a0a0468b6a0ac2cb0a9f2f36b	3d3d1a2f671d1a9fa0cb7f85c2d7f386	delivered	2018-04-11 10:48:14	2018-02-27 04:04:16	2018-05-09 15:48:00	2017-05-16 00:00:00	2017-12-20 00:00:00
freq	1	1	96478	3	9	47	3	1652

Building RFM model

```
# 查看处理后的数据
print("处理后数据形状:", df.shape)
print("\n列名:", df.columns.tolist)

# 基本统计信息
print("数值型变量统计:")
df.describe()

# 分类变量统计
print("分类变量统计:")
df.describe(include=['object'])

# 时间范围分析
print("时间范围:")
print("最早订单时间:", df['order_purchase_timestamp'].min())
print("最晚订单时间:", df['order_purchase_timestamp'].max())

# 订单状态分布
print("订单状态分布:")
print(df['order_status'].value_counts)
```

```
处理后数据形状: (99441, 8)

列名: <bound method Index.tolist of Index(['order_id', 'customer_id', 'order_status',
       'order_purchase_timestamp', 'order_approved_at',
       'order_delivered_carrier date', 'order_delivered_customer date',
       'order_estimated_delivery date'],
      dtype=object)>
数值型变量统计:
分类变量统计:
时间范围:
最早订单时间: 2016-09-04 21:15:19
最晚订单时间: 2018-10-17 17:30:18
订单状态分布:
order_status
delivered      96478
shipped         1107
canceled         625
unavailable      609
invoiced         314
processing       301
created            5
approved           2
Name: count, dtocs: int64
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier date	order_delivered_customer date	order_estimated_delivery date
count	99441	99441	99441	99441	99281	98966	96476	99441
unique	99441	99441	8	98875	90733	81018	95834	459
top	73c2643a0a0468b6a0ac2cb0a9f2f36b	3d3d1a2f671d1a9fa0cb7f85c2d7f386	delivered	2018-04-11 10:48:14	2018-02-27 04:04:16	2018-05-09 15:48:00	2017-05-16 00:00:00	2017-12-20 00:00:00
freq	1	1	96478	3	9	47	3	1652

```
# 计算客户最近一次购买时间
print("计算客户最近一次购买时间...")
customer_last_purchase = df.groupby('customer_id')['order_purchase_timestamp'].max()
print("客户数量:", len(customer_last_purchase))

# 计算每个客户的订单数
print("计算客户订单数...")
customer_orders = df.groupby('customer_id')['order_id'].count()
print("平均订单数:", customer_orders.mean())

# 计算每个客户的总消费
print("计算客户总消费...")
customer_spending = df.groupby('customer_id')['price'].sum()
print("平均消费:", customer_spending.mean())

# 合并数据
print("合并数据...")
rfm = pd.DataFrame({
    'recency': (pd.to_datetime('2018-10-17') - customer_last_purchase).dt.days,
    'frequency': customer_orders,
    'monetary': customer_spending
})
print("RFM数据形状:", rfm.shape)

# 查看RFM数据的基本统计
print("RFM数据统计:")
rfm.describe()

# 保存RFM数据
rfm.to_csv('rfm_data.csv')
print("RFM数据已保存")
```

```
计算客户最近一次购买时间...
客户数量: 96096
计算客户订单数...
平均订单数: 1.0341109366368366
计算客户总消费...
平均消费: 160.99414229752066
合并数据...
RFM数据形状: (96096, 3)
RFM数据统计:
RFM数据已保存
```

	recency	frequency	monetary
count	96096.000000	96096.000000	96096.000000
mean	237.156700	1.034111	160.994142
std	152.826851	0.209382	219.454966
min	0.000000	1.000000	9.590000
25%	114.000000	1.000000	63.049000
50%	219.000000	1.000000	112.846000
75%	346.000000	1.000000	196.004000
max	714.000000	15.000000	13664.080000

```
# 查看RFM数据的基本信息
print("RFM数据信息:")
rfm.info()

# 查看缺失值
print("缺失值:")
rfm.isnull().sum()

# 查看唯一值数量
print("唯一值数量:")
rfm.nunique()

# 查看数值型变量的分布
print("数值型变量分布:")
rfm.describe()
```

```
RFM数据信息:
<class pandas.core.frame.DataFrame>
Index: 96096 entries, 0000366f3b9a7992bf8c76cfdf3221e2 to 9ffb9a2ff96bd8ca301a0a71cb8f27ab
Data columns (total 3 columns):
 recency    96096 non-null int64
 frequency  96096 non-null int64
 monetary   96096 non-null float64
dtypes: int64(2), float64(1)
memory usage: 2.9 MB
缺失值:
唯一值数量:
数值型变量分布:
```

	recency	frequency	monetary
count	96096.000000	96096.000000	96096.000000
mean	237.156700	1.034111	160.994142
std	152.826851	0.209382	219.454966
min	0.000000	1.000000	9.590000
25%	114.000000	1.000000	63.049000
50%	219.000000	1.000000	112.846000
75%	346.000000	1.000000	196.004000
max	714.000000	15.000000	13664.080000

```
# 创建子图
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# recency分布
axes[0].hist(rfm['recency'], bins=50, alpha=0.7)
axes[0].set_title('recency分布')
axes[0].set_xlabel('天数')
axes[0].set_ylabel('客户数')

# frequency分布
axes[1].hist(rfm['frequency'], bins=50, alpha=0.7)
axes[1].set_title('frequency分布')
axes[1].set_xlabel('订单数')
axes[1].set_ylabel('客户数')

# monetary分布
axes[2].hist(rfm['monetary'], bins=50, alpha=0.7)
axes[2].set_title('monetary分布')
axes[2].set_xlabel('消费金额')
axes[2].set_ylabel('客户数')

plt.tight_layout()
plt.show()
```

```
# 查看RFM数据的基本信息
print("RFM数据信息:")
rfm.info()

# 查看缺失值
print("缺失值:")
rfm.isnull().sum()

# 查看唯一值数量
print("唯一值数量:")
rfm.nunique()

# 查看数值型变量的分布
print("数值型变量分布:")
rfm.describe()
```

```
RFM数据信息:
<class pandas.core.frame.DataFrame>
Index: 96096 entries, 0000366f3b9a7992bf8c76cfdf3221e2 to 9ffb9a2ff96bd8ca301a0a71cb8f27ab
Data columns (total 3 columns):
 recency    96096 non-null int64
 frequency  96096 non-null int64
 monetary   96096 non-null float64
dtypes: int64(2), float64(1)
memory usage: 2.9 MB
缺失值:
唯一值数量:
数值型变量分布:
```

	recency	frequency	monetary
count	96096.000000	96096.000000	96096.000000
mean	237.156700	1.034111	160.994142
std	152.826851	0.209382	219.454966
min	0.000000	1.000000	9.590000
25%	114.000000	1.000000	63.049000
50%	219.000000	1.000000	112.846000
75%	346.000000	1.000000	196.004000
max	714.000000	15.000000	13664.080000

```
# 创建子图
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# recency分布
axes[0].hist(rfm['recency'], bins=50, alpha=0.7)
axes[0].set_title('recency分布')
axes[0].set_xlabel('天数')
axes[0].set_ylabel('客户数')

# frequency分布
axes[1].hist(rfm['frequency'], bins=50, alpha=0.7)
axes[1].set_title('frequency分布')
axes[1].set_xlabel('订单数')
axes[1].set_ylabel('客户数')

# monetary分布
axes[2].hist(rfm['monetary'], bins=50, alpha=0.7)
axes[2].set_title('monetary分布')
axes[2].set_xlabel('消费金额')
axes[2].set_ylabel('客户数')

plt.tight_layout()
plt.show()
```



```
# 计算RFM分数
print="计算RFM分数..."
rfm['R_score'] = pd.qcut(rfm['recency'], 5, labels=[5, 4, 3, 2, 1])
rfm['F_score'] = pd.qcut(rfm['frequency'].rank(method='first'), 5, labels=[1, 2, 3, 4, 5])
rfm['M_score'] = pd.qcut(rfm['monetary'], 5, labels=[1, 2, 3, 4, 5])

print="RFM分数统计:"
print(rfm[['R_score', 'F_score', 'M_score']].describe())

# 查看RFM分数的分布
print="RFM分数分布:"
print(rfm[['R_score', 'F_score', 'M_score']].value_counts())
```

```
计算RFM分数...
RFM分数统计:
       R_score       F_score       M_score
count  96096.0  96096.000000  96096.000000
mean       3.0      1.000000      2.000000
std        1.4      0.000000      1.414214
min        1.0      1.000000      1.000000
25%        2.0      1.000000      1.000000
50%        3.0      1.000000      2.000000
75%        4.0      1.000000      3.000000
max        5.0      1.000000      5.000000
RFM分数分布:
R_score  F_score  M_score
1.0      1.0      1.0        19246
2.0      1.0      1.0        19246
3.0      1.0      1.0        19246
4.0      1.0      1.0        19246
5.0      1.0      1.0        19246
Name: count, dtype: int64
```

```
# 创建RFM分数组合
print="创建RFM分数组合..."
rfm['RFM_score'] = rfm['R_score'].astype(str) + rfm['F_score'].astype(str) + rfm['M_score'].astype(str)

print="RFM分数组合统计:"
print(rfm['RFM_score'].value_counts())

# 查看前10个最常见的RFM组合
print="最常见的RFM组合:"
print(rfm['RFM_score'].value_counts().head(10))
```

```
创建RFM分数组合...
RFM分数组合统计:
RFM_score
111    19246
211    19246
311    19246
411    19246
511    19246
Name: count, dtype: int64
最常见的RFM组合:
RFM_score
111    19246
211    19246
311    19246
411    19246
511    19246
Name: count, dtype: int64
```

```
# 定义客户分群函数
def customer_segment(rfm):
    if rfm['R_score'] >= 4 and rfm['F_score'] >= 4 and rfm['M_score'] >= 4:
        return '高价值客户'
    elif rfm['R_score'] >= 4 and rfm['F_score'] >= 3:
        return '潜力客户'
    elif rfm['R_score'] >= 3 and rfm['F_score'] >= 3:
        return '普通客户'
    elif rfm['R_score'] >= 2 and rfm['F_score'] >= 2:
        return '需关注客户'
    elif rfm['R_score'] >= 1 and rfm['F_score'] >= 1:
        return '流失客户'
    else:
        return '其他'

# 应用分群函数
print="应用客户分群..."
rfm['customer_segment'] = rfm.apply(customer_segment, axis=1)

print="客户分群统计:"
print(rfm['customer_segment'].value_counts)

# 查看各群体的特征
print="各群体特征:"
print(rfm.groupby('customer_segment')[['recency', 'frequency', 'monetary']].mean())
```

```
应用客户分群...
客户分群统计:
customer_segment
流失客户    19246
其他      19246
普通客户    19246
需关注客户   19246
潜力客户    19246
Name: count, dtype: int64
各群体特征:
                 recency  frequency  monetary
customer_segment
流失客户            0.000        1.0     9.590
其他                0.000        1.0    63.049
普通客户            0.000        1.0   112.846
需关注客户           0.000        1.0   196.004
潜力客户            0.000        1.0 13664.080
```

```
# 计算肘部法则数据
print="计算肘部法则数据..."
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

inertia = []
silhouette = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(rfm[['recency', 'frequency', 'monetary']])
    inertia.append(kmeans.inertia_)
    silhouette.append(silhouette_score(rfm[['recency', 'frequency', 'monetary']], kmeans.labels_))

print="肘部法则数据已计算"

# 绘制肘部法则图
plt.figure(figure="肘部法则分析", figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(2, 11), inertia, 'bo-')
plt.title('肘部法则')
plt.xlabel('聚类数')
plt.ylabel('惯性')

plt.subplot(1, 2, 2)
plt.plot(range(2, 11), silhouette, 'ro-')
plt.title('轮廓系数')
plt.xlabel('聚类数')
plt.ylabel('轮廓系数')

plt.tight_figure()
plt.show()
```

```
计算肘部法则数据...
肘部法则数据已计算
```



```
# 设置聚类数
print="设置聚类数..."
n_clusters = 5

# 执行K-means聚类
print="执行K-means聚类..."
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
rfm['cluster'] = kmeans.fit_predict(rfm[['recency', 'frequency', 'monetary']])

print="聚类结果统计:"
print(rfm['cluster'].value_counts)

# 查看各聚类的特征
print="各聚类特征:"
print(rfm.groupby('cluster')[['recency', 'frequency', 'monetary']].mean)
```

```
设置聚类数...
执行K-means聚类...
聚类结果统计:
cluster
0    19246
1    19246
2    19246
3    19246
4    19246
Name: count, dtype: int64
各聚类特征:
         recency  frequency  monetary
cluster
0          0.000        1.0     9.590
1          0.000        1.0    63.049
2          0.000        1.0   112.846
3          0.000        1.0   196.004
4          0.000        1.0 13664.080
```

```
# 创建聚类分析可视化
print="创建聚类分析可视化..."
fig = plt.figure(figure="聚类分析", figsize=(15, 10))

# 1. 聚类分布
plt.subplot(2, 3, 1)
cluster_counts = rfm['cluster'].value_counts()
plt.bar(cluster_counts.index, cluster_counts.values)
plt.title('聚类分布')
plt.xlabel('聚类')
plt.ylabel('客户数')

# 2. 3D散点图
from mpl_toolkits.mpl_toolkits.mplot3d import Axes3D
ax = fig.add_subplot(2, 3, 2, projection='3d')
scatter = ax.scatter(rfm['recency'], rfm['frequency'], rfm['monetary'],
                     c=rfm['cluster'], cmap='viridis')
ax.set_title('3D聚类')
ax.set_xlabel('recency')
ax.set_ylabel('frequency')
ax.set_zlabel('monetary')

# 3. 箱线图
plt.subplot(2, 3, 3)
rfm.boxplot(column=['recency', 'frequency', 'monetary'], by='cluster')
plt.title('各聚类特征')
plt.suptitle('')

plt.tight_figure()
plt.show()
```

```
创建聚类分析可视化...
```



```
# 创建RFM热力图
print="创建RFM热力图..."
plt.figure(figure="RFM热力图", figsize=(8, 6))

# 计算各聚类的平均值
cluster_mean = rfm.groupby('cluster')[['recency', 'frequency', 'monetary']].mean()

# 绘制热力图
import seaborn as sns
sns.heatmap(cluster_mean.T, annot=True, cmap='viridis')
plt.title('RFM热力图')
plt.xlabel('聚类')
plt.ylabel('特征')

plt.tight_figure()
plt.show()
```

```
创建RFM热力图...
```



```
# 创建客户群体特征分析
print="创建客户群体特征分析..."
customer_profile = rfm.groupby('customer_segment')[['recency', 'frequency', 'monetary']].mean()

print="客户群体特征:"
print(customer_profile)

# 计算各群体的客户数
print="各群体客户数:"
print(rfm['customer_segment'].value_counts)

# 计算各群体的消费贡献
print="各群体消费贡献:"
print(rfm.groupby('customer_segment')['monetary'].sum / rfm['monetary'].sum * 100)
```

```
创建客户群体特征分析...
客户群体特征:
                 recency  frequency  monetary
customer_segment
流失客户            0.000        1.0     9.590
其他                0.000        1.0    63.049
普通客户            0.000        1.0   112.846
需关注客户           0.000        1.0   196.004
潜力客户            0.000        1.0 13664.080
各群体客户数:
customer_segment
流失客户    19246
其他      19246
普通客户    19246
需关注客户   19246
潜力客户    19246
Name: count, dtype: int64
各群体消费贡献:
customer_segment
流失客户    0.000000
其他      0.000000
普通客户    0.000000
需关注客户   0.000000
潜力客户   100.000000
Name: monetary, dtype: float64
```

```
# 创建营销策略建议
print="创建营销策略建议..."
strategy = {
    '高价值客户': '提供VIP服务，推荐高价值产品，建立忠诚度计划',
    '潜力客户': '提供个性化推荐，交叉销售，提升客户价值',
    '普通客户': '维持现有关系，定期营销，防止流失',
    '需关注客户': '重新激活，提供优惠，了解流失原因',
    '流失客户': '挽回营销，调查流失原因，改进服务'
}

print="营销策略建议:"
for segment, strategy in strategy.items():
    print(f"{segment}: {strategy}")

# 计算各策略的预期效果
print="计算预期效果..."
expected_impact = {
    '高价值客户': '保持高价值客户，提升忠诚度',
    '潜力客户': '提升客户价值，增加交叉销售',
    '普通客户': '维持现有关系，防止流失',
    '需关注客户': '重新激活，减少流失',
    '流失客户': '挽回部分客户，改进服务'
}

print="预期效果:"
for segment, impact in expected_impact.items():
    print(f"{segment}: {impact}")
```

```
创建营销策略建议...
营销策略建议:
高价值客户: 提供VIP服务，推荐高价值产品，建立忠诚度计划
潜力客户: 提供个性化推荐，交叉销售，提升客户价值
普通客户: 维持现有关系，定期营销，防止流失
需关注客户: 重新激活，提供优惠，了解流失原因
流失客户: 挽回营销，调查流失原因，改进服务
计算预期效果...
预期效果:
高价值客户: 保持高价值客户，提升忠诚度
潜力客户: 提升客户价值，增加交叉销售
普通客户: 维持现有关系，防止流失
需关注客户: 重新激活，减少流失
流失客户: 挽回部分客户，改进服务
```

Conclusion: Form k-means, we have only five groups. We can have different strategies for different groups. High-P and M contribute to always our important customer.Some Customer Retention Strategies